ATTABL

Système de Terminologie Contextuelle

par Segment d’Activité

V2 — Mise à jour post-implémentation

Architecture technique, état d’implémentation
et plan d’action restant

Mars 2026 • Document Confidentiel • V2

Changelog V1 → V2

Ce document est la mise à jour du document ARCHITECTURE-TERMINOLOGIE-CONTEXTUELLE.docx (V1, 12 mars 2026) après analyse de l'état réel du codebase.

Ce qui a été implémenté depuis la V1

- **segment-terms.ts** — Créé avec une approche améliorée : au lieu de strings hardcodées (comme proposé dans V1), utilise des clés i18n résolues via le namespace **segment** de next-intl. Meilleur pour le multi-langue.
- **segment-features.ts** — Créé comme proposé, plus fonction **getHiddenNav()** pour le filtrage sidebar (pas dans V1).
- **useSegmentTerms.ts** — Hook créé, utilise useMemo + useTranslations('segment') pour performance optimale.
- **useSegmentFeatures.ts** — Hook créé comme proposé.
- **EstablishmentStep.tsx** — Les 10 types sont en place (les 4 nouveaux : retail, boutique, pharmacy, salon) + champs spécifiques par type (tableCount, roomCount, starRating, etc.).
- **AdminSidebar.tsx** — Utilise getHiddenNav() + labels segment-aware. Groupes kitchen/service masqués pour retail/services.
- **AdminBottomNav.tsx** — Même logique segment que le sidebar (non prévu dans V1 — bonus).
- **ItemsClient.tsx** — Utilise useSegmentTerms() pour les labels dynamiques.
- **Onboarding** — page.tsx, LaunchStep.tsx, MenuStep.tsx utilisent getSegmentFeatures/getSegmentFamily.
- **Namespace i18n « segment »** — Créé dans fr-FR.json ET en-US.json avec 20 clés × 4 familles.
- **Locales simplifiées** — De 8 locales à 2 (fr-FR, en-US). La V1 mentionnait « 8 fichiers i18n » — corrigé.

Ce qui a changé par rapport à la V1

- **Architecture segment-terms :** La V1 proposait des strings FR hardcodées dans segment-terms.ts (« Plat », « Article »). L'implémentation réelle utilise des clés i18n (**food.item**, **retail.item**) résolues par next-intl. C'est mieux : ça supporte nativement le multi-langue sans dupliquer les termes.
- **Champs onboarding par type :** V1 ne prévoyait pas de champs spécifiques par type d'établissement. L'implémentation actuelle a des champs adaptés (tableCount pour resto, roomCount/starRating pour hôtel, registerCount pour retail, etc.).
- **getHiddenNav() :** Fonction ajoutée dans segment-features.ts pour pré-calculer les groupes et paths à masquer. Plus performant que le pattern proposé en V1.

Ce qui reste à faire

- **Migration complète des composants :** Seul ItemsClient.tsx utilise useSegmentTerms(). Il reste les composants POS, KDS, Orders, Dishes, Settings à migrer.
- **Pages marketing verticales :** Aucune page marketing par segment n'a été créée (ce sujet est traité dans le document REBRANDING-LANDING-PAGE-ATTABL.docx).
- **Tests E2E par segment :** Aucun test E2E spécifique au système de segments n'existe encore.
- **Pont landing page → segment :** La landing page ne propose pas encore d'auto-identification par segment avant inscription (prévu dans le doc rebranding, Contre-Proposition B hero).

1. Le Principe : Storytelling Intelligent par Segment

Le problème n'est pas d'avoir des termes restaurant dans ATTABL — c'est qu'un commerçant retail voit des termes restaurant quand il utilise la plateforme. La solution n'est pas de tout rendre générique (« établissement » partout), c'est de rendre la terminologie contextuelle et intelligente.

> *Chaque segment d'activité a son propre vocabulaire. Un restaurateur voit « Plats », « Serveur », « Cuisine ». Un commerçant retail voit « Articles », « Vendeur », « Préparation ». Un salon voit « Prestations », « Coiffeur », « Atelier ».*

1.1 Comment ça fonctionne (vue utilisateur)

1. À l'onboarding, le tenant choisit son type d'activité parmi 10 types
2. Ce choix détermine un « segment family » (food, hospitality, retail, services) stocké via establishment_type
3. Le segment family mappe vers des clés i18n dans le namespace « segment » de next-intl
4. Le hook useSegmentTerms() résout ces clés et retourne les termes traduits
5. Le hook useSegmentFeatures() détermine quels modules sont visibles

1.2 Ce qui ne change PAS

- Le cœur technique (routes API, services, DB) ne change pas
- Les composants React restent les mêmes — seuls les textes affichés changent
- Le système next-intl existant n'est pas modifié — on ajoute un namespace « segment » par-dessus
- Le tenant peut changer son type dans les paramètres → la terminologie s'adapte en temps réel

2. Architecture Technique

2.1 Système i18n actuel

Framework : next-intl (getRequestConfig + useTranslations)

Fichiers : src/messages/fr-FR.json et en-US.json (2 locales)

Chargement : cookie NEXT_LOCALE → import dynamique du JSON

Tenant type : establishment_type dans la table tenants, accessible via useTenant()

2.2 Architecture implémentée

Contrairement à la V1 qui proposait des strings hardcodées en français, l'implémentation utilise le système i18n pour supporter nativement le multi-langue.

Fichier : src/lib/segment-terms.ts

Ce fichier définit les types, les familles de segments, et génère des clés i18n (pas des strings FR directes).

```
type EstablishmentSegment = 'restaurant' | 'hotel' | 'bar' | 'cafe' | 'fastfood' |
'retail' | 'boutique' | 'pharmacy' | 'salon' | 'other'

type SegmentFamily = 'food' | 'hospitality' | 'retail' | 'services'

getSegmentFamily('retail') → 'retail'

getSegmentTermKeys('retail') → { item: 'retail.item', worker: 'retail.worker', ...
}
```

Les clés sont pré-calculées via TERM_KEYS_BY_FAMILY pour éviter toute allocation à chaque appel — performance optimale.

Fichier : src/lib/segment-features.ts

Définit les modules visibles par famille de segment + le filtrage de navigation pré-calculé.

```
getSegmentFeatures('salon') → { showKds: false, showTables: false, showService:
false, showRecipes: false, showQrCodes: false }

getHiddenNav('retail') → { groupIds: Set{'kitchen','service'}, itemPaths: Set{} }
```

Hooks React

useSegmentTerms() — Récupère le type du tenant via useTenant(), résout les clés i18n via useTranslations('segment'), retourne les termes traduits. Memoïsé via useMemo.

useSegmentFeatures() — Récupère le type du tenant, retourne les feature flags booleéns.

3. Mapping Terminologique Complet

Voici le mapping complet tel qu'il existe dans fr-FR.json (namespace « segment ») et en-US.json :

Français (fr-FR)

Clé	Food	Hospitality	Retail	Services
item	Plat	Plat	Article	Prestation
items	Plats	Plats	Articles	Prestations
addItem	Ajouter un plat	Ajouter un plat	Ajouter un article	Ajouter une prestation
searchItem	Rechercher un plat	Rechercher un plat	Rechercher un article	Rechercher une prestation
worker	Serveur	Réceptionniste	Vendeur	Coiffeur
workers	Serveurs	Réceptionnistes	Vendeurs	Coiffeurs
production	Cuisine	Cuisine	Préparation	Atelier
productionKds	Cuisine (KDS)	Cuisine (KDS)	Prépa (KDS)	Atelier (KDS)
catalog	Carte / Menu	Room Service	Catalogue	Catalogue
catalogs	Cartes & Menus	Room Services	Catalogues	Catalogues
recipe	Fiche technique	Fiche technique	Fiche produit	Fiche prestation
recipes	Fiches techniques	Fiches techniques	Fiches produits	Fiches prestations
allLocations	Tous les restaurants	Tous les hôtels	Toutes les boutiques	Tous les salons
identityLabel	Identité du restaurant	Identité de l'hôtel	Identité de la boutique	Identité du salon
emailPlaceholder	jean@restaurant.com	jean@monhotel.com	jean@maboutique.com	jean@monsalon.com
descPlaceholder	Cuisine locale et...	Hôtel de charme...	Décrivez votre boutique...	Salon de coiffure...
roleChef	Chef cuisinier	Chef cuisinier	Resp. stock	Resp. technique
roleWaiter	Serveur	Réceptionniste	Vendeur	Coiffeur / Esthéticien
domainPlaceholder	menu.monrestaurant.com	menu.monhotel.com	shop.maboutique.com	rdv.monsalon.com

4. Modules à Visibilité Conditionnelle

La sidebar et le bottom nav s'adaptent automatiquement au segment du tenant. Voici la matrice complète :

Module / Nav	Food	Hospitality	Retail	Services
Tableau de bord	☐	☐	☐	☐
Commandes	☐	☐	☐	☐
Catalogue / Menus	☐	☐	☐	☐
Articles / Plats	☐	☐	☐	☐
Cuisine (KDS)	☐	☐	☐	☐
Caisse (POS)	☐	☐	☐	☐
Tables / Zones	☐	☐	☐	☐
Service (assignments)	☐	☐	☐	☐
Inventaire / Stock	☐	☐	☐	☐
Fiches techniques	☐	☐	☐	☐
Fournisseurs	☐	☐	☐	☐
Rapports / Analytics	☐	☐	☐	☐
QR Codes	☐	☐	☐	☐
Coupons / Promotions	☐	☐	☐	☐

Implémentation dans le code

AdminSidebar.tsx et AdminBottomNav.tsx utilisent getHiddenNav() pour filtrer dynamiquement les éléments de navigation. Les groupes « kitchen » et « service » sont masqués pour retail et services. Le path « /recipes » est masqué pour services.

Les labels sont également adaptés : « Plats » devient « Articles » pour retail, « Cuisine (KDS) » devient « Prépa (KDS) », etc.

5. Onboarding Adaptatif

5.1 Types d'établissement

Les 10 types sont implémentés dans EstablishmentStep.tsx avec des champs spécifiques par type :

ID	Label FR	Famille	Icône	Champs spécifiques
restaurant	Restaurant	food	UtensilsCrossed	tableCount, totalCapacity
hotel	Hôtel	hospitality	Hotel	roomCount, starRating, hasRestaurant
bar	Bar	food	Wine	tableCount, hasTerrace
cafe	Café	food	Coffee	seatCount, hasWifi
fastfood	Fast-food	food	Flame	registerCount, hasDelivery
retail	Épicerie / Supermarché	retail	ShoppingCart	registerCount, hasDelivery
boutique	Boutique / Mode	retail	Store	registerCount, hasDelivery
pharmacy	Pharmacie	retail	Heart	registerCount, hasDelivery
salon	Salon (coiffure, beauté)	services	Scissors	totalCapacity
other	Autre commerce	retail	Building2	tableCount

5.2 Étapes adaptatives par segment

L'onboarding adapte ses étapes selon la famille du segment choisi :

Étape	Food / Hospitality	Retail	Services
Type	Sélection + champs spécifiques	Sélection + champs spécifiques	Sélection + champs spécifiques
Branding	Logo, couleurs, description	Logo, couleurs, description	Logo, couleurs, description
Tables	Config zones & tables	Masqué (showTables: false)	Masqué (showTables: false)
Menu / Catalogue	« Créez vos premiers plats »	« Créez vos premiers articles »	« Créez vos premières prestations »
QR / Lancement	QR codes (tables)	QR codes (produits)	Lien de partage uniquement

L'étape Tables est conditionnée par segmentFeatures.showTables dans onboarding/page.tsx. L'étape QR est conditionnée par segmentFeatures.showQrCodes dans LaunchStep.tsx.

6. Cartographie des Fichiers

6.1 Fichiers créés

Fichier	Rôle	Statut
src/lib/segment-terms.ts	Mapping types → familles → clés i18n	□ FAIT
src/lib/segment-features.ts	Feature flags + nav caché par famille	□ FAIT
src/hooks/useSegmentTerms.ts	Hook React pour termes traduits	□ FAIT
src/hooks/useSegmentFeatures.ts	Hook React pour feature flags	□ FAIT
src/messages/fr-FR.json (namespace segment)	20 clés × 4 familles en français	□ FAIT
src/messages/en-US.json (namespace segment)	20 clés × 4 familles en anglais	□ FAIT

6.2 Fichiers modifiés

Fichier	Modification	Statut
src/components/onboarding/EstablishmentStep.tsx	10 types + champs spécifiques	□ FAIT
src/components/admin/AdminSidebar.tsx	getHiddenNav() + labels segment	□ FAIT
src/components/admin/AdminBottomNav.tsx	Même logique que sidebar	□ FAIT
src/components/admin/ItemsClient.tsx	useSegmentTerms() pour labels	□ FAIT
src/app/onboarding/page.tsx	getSegmentFeatures() pour étapes	□ FAIT
src/components/onboarding/LaunchStep.tsx	getSegmentFeatures() pour QR	□ FAIT
src/components/onboarding/MenuStep.tsx	getSegmentFamily() pour contexte	□ FAIT
src/types/admin.types.ts	WizardItem + establishment_type typé	□ FAIT
src/services/tenant.service.ts	Update establishment_type	□ FAIT
src/i18n/config.ts	Simplifié à 2 locales + LOCALE_LABELS	□ FAIT

6.3 Fichiers restant à migrer

Fichier / Zone	Migration nécessaire	Priorité
Composants utilisant t('dishes.*')	Remplacer par terms.item / terms.addItem	Haute

Composants utilisant t('orders.allRestaurants')	Remplacer par terms.allLocations	Haute
POS / Caisse	Labels segment-aware pour les actions	Haute
KDS / Cuisine	Labels : « Cuisine » → terms.production	Moyenne
Settings / Identity	terms.identityLabel au lieu de hardcodé	Haute
Pages marketing par segment	Création de pages verticales	Moyenne
Tests E2E par segment	Scénarios retail, salon, hotel	Haute

7. Feuille de Route Mise à Jour

La feuille de route originale prévoyait 5 semaines (16 jours-homme). Voici l'état d'avancement :

#	Action	Effort	Statut
1	Créer segment-terms.ts + segment-features.ts	1 jour	☐ FAIT
2	Créer useSegmentTerms + useSegmentFeatures hooks	0.5 jour	☐ FAIT
3	Ajouter les 10 types dans onboarding + admin.types	1 jour	☐ FAIT
4	Migrer Sidebar + BottomNav vers segment	1 jour	☐ FAIT
5	Migrer ItemsClient vers useSegmentTerms	0.5 jour	☐ FAIT
6	Migrer POS, KDS, Dishes, Orders vers useSegmentTerms	3 jours	☐ TODO
7	Migrer Settings vers useSegmentTerms	1 jour	☐ TODO
8	Mettre à jour SEO metadata + pages marketing	2 jours	☐ TODO
9	Créer pages marketing verticales par segment	3 jours	☐ TODO
10	Tests E2E par segment + corrections	2 jours	☐ TODO

Progression : 5 actions sur 10 complétées (4 jours sur 16 jours-homme). Reste : ~11 jours-homme de travail (actions 6 à 10).

7.1 Priorités immédiates (prochaines 2 semaines)

- **Action 6 — Migration POS/KDS/Dishes/Orders :** C'est le plus gros chantier restant. Chaque composant qui affiche « Plat », « Restaurant », « Serveur » doit utiliser useSegmentTerms() à la place.
- **Action 7 — Settings :** La page Identité affiche encore « Identité du restaurant » en dur. Doit utiliser terms.identityLabel.
- **Action 10 — Tests E2E :** Créer au minimum un scénario E2E par famille (food + retail + services) pour valider que le système fonctionne de bout en bout.

7.2 Synergies avec le rebranding landing page

Le document REBRANDING-LANDING-PAGE-ATTABL.docx prévoit des évolutions marketing qui doivent s'articuler avec le système de terminologie contextuelle :

- **Contre-Proposition B du Hero :** Les pastilles cliquables (Restaurant, Boutique, Salon, Hôtel) dans le hero créent un pont entre la landing page et le système de segments. Le visiteur s'auto-identifie avant même l'inscription.

- **Section Segments (Prop A ou B) :** Les segments affichés sur la landing page doivent correspondre exactement aux 10 types + 4 familles définis dans segment-terms.ts.
- **Parcours signup pré-segmenté :** Si un visiteur clique sur « Découvrir ATTABL pour Boutiques », l'onboarding devrait pré-sélectionner le type « boutique ». Cela nécessite un paramètre URL (ex: ?segment=boutique) lu par EstablishmentStep.tsx.
- **Pages marketing verticales (Action 9) :** Chaque page verticale (/for/restaurants, /for/retail, /for/salons) doit utiliser le vocabulaire du segment correspondant, aligné sur les termes définis dans segment-terms.ts.

8. Conclusion

Le système de terminologie contextuelle est en bonne voie. Les fondations sont solides : l'architecture (segment-terms, segment-features, hooks) est en place, le mapping i18n est complet en 2 langues, l'onboarding et la navigation sont adaptés.

L'implémentation a même amélioré la proposition initiale (V1) sur plusieurs points : utilisation de clés i18n au lieu de strings hardcodées, pré-calcul des termes et de la navigation pour la performance, champs d'onboarding spécifiques par type d'établissement.

Le travail restant est principalement de la migration de composants existants (étapes 6-7) et de la création de contenu marketing (8-9). L'effort est estimé à 11 jours-homme.

> *Le résultat : ATTABL devient une plateforme où chaque professionnel — restaurateur, hôtelier, commerçant, coiffeur — se sent chez lui dès la première connexion.*

Document généré le 15 mars 2026 (V2) — Basé sur l'analyse du codebase ATTABL (segment-terms.ts, segment-features.ts, 2 fichiers i18n, 12 fichiers référençant establishment_type, AdminSidebar, AdminBottomNav, ItemsClient, onboarding).